
Towards Practical Defect-Focused Automated Code Review

Junyi Lu^{1 2 †} Lili Jiang³ Xiaojia Li³ Jianbing Fang^{4 †} Fengjun Zhang¹ Li Yang¹ Chun Zuo⁵

Abstract

The complexity of code reviews has driven efforts to automate review comments, but prior approaches oversimplify this task by treating it as snippet-level code-to-text generation and relying on text similarity metrics like BLEU for evaluation. These methods overlook repository context, real-world merge request evaluation, and defect detection, limiting their practicality. To address these issues, we explore the full automation pipeline within the online recommendation service of a company with nearly 400 million daily active users, analyzing industry-grade C++ codebases comprising hundreds of thousands of lines of code. We identify four key challenges: ❶ capturing relevant context, ❷ improving key bug inclusion (KBI), ❸ reducing false alarm rates (FAR), and ❹ integrating human workflows. To tackle these, we propose ❶ code slicing algorithms for context extraction, ❷ a multi-role LLM framework for KBI, ❸ a filtering mechanism for FAR reduction, and ❹ a novel prompt design for better human interaction. Our approach, validated on real-world merge requests from historical fault reports, achieves a 2× improvement over standard LLMs and a 10× gain over previous baselines. While the presented results focus on C++, the underlying framework design leverages language-agnostic principles (e.g., AST-based analysis), suggesting potential for broader applicability.

1. Introduction

Code review is essential for improving code quality and detecting defects (Fagan, 2002). Modern Code Review

(MCR) is widely used in open-source (Rigby et al., 2008; 2014; Rigby & Bird, 2013) and industrial settings (Sadowski et al., 2018; Shan et al., 2022), typically involving: (A) code submission, (B) reviewer examination, (C) feedback, and (D) developer revisions.

Despite its benefits, MCR is labor-intensive and time-consuming (Yang et al., 2016), driving research toward automated review comment generation. Existing methods—whether retrieval-based (Gupta & Sundaresan, 2018; Siow et al., 2020; Hong et al., 2022) or deep-learning-driven (Tufano et al., 2021; 2022; Li et al., 2022b;a; Lin et al., 2023; Lu et al., 2023)—often frame it as a **snippet-level code-to-text** task. However, this oversimplification diverges from the core goal of reviewers: detecting defects (Bacchelli & Bird, 2013) (see Section A). Furthermore, current evaluations rely excessively on textual similarity metrics (e.g., BLEU (Papineni et al., 2002), ROUGE (Lin, 2004)), which fail to measure real-world effectiveness (Lu et al., 2025).

Challenges. To address these limitations, we investigate a full code review pipeline within a real-world online service (Figure 1). Our system integrates with an internal DevOps platform, generating review reports, filtering comments, and aligning them with code lines. A detailed description of this real-world workflow integration, designed for seamless adoption by developers, is provided in Appendix B. This deployment reveals four key challenges (Appendix C):

Capturing Proper Code Context: Effective review requires analyzing dependencies beyond the immediate diff hunk (e.g., variable declarations or method calls). However, excessively long inputs degrade LLM performance, necessitating efficient context extraction.

Improving Key Bug Inclusion (KBI): The goal of automated review is to detect critical defects, yet existing methods rely on textual similarity metrics, which fail to measure defect detection capability. More robust evaluation methods, such as Key-Bug Inclusion (KBI), are needed.

Reducing False Alarm Rates (FAR): Generative models often produce irrelevant or overly strict comments (e.g., nit-picks, hallucinations), burdening developers. A robust filtering mechanism is required to reduce false positives and enhance signal-to-noise ratio.

Human-Centric Workflow Integration: Practical review tools

[†] Work done during the internship or tenure at Kuaishou Technology. ¹Laboratory of Precise Computing, Institute of Software, Chinese Academy of Sciences, Beijing, China ²University of Chinese Academy of Sciences, Beijing, China ³Kuaishou Technology, Beijing, China ⁴Independent Researcher ⁵Sinsoft Company Limited, Beijing, China. Correspondence to: Li Yang <yangli2017@iscas.ac.cn>.



Figure 1. The code review automation pipeline integrated into the online service.

must seamlessly integrate into developers’ workflows, ensuring comment alignment with code lines while minimizing cognitive overhead. Existing solutions often overlook this critical usability aspect.

Our Approach. To address these challenges, we propose:

- ❶ A static analysis system using code slicing to extract relevant context.
- ❷ A multi-role LLM framework with chain-of-thought reasoning to enhance defect detection.
- ❸ A filtering mechanism to eliminate false positive nitpicks and hallucinations.
- ❹ A line-aware prompt design for precise comment placement.

Evaluation. We validate our framework on real-world system failures, including historical core dumps and fault reports that caused significant financial losses. We evaluate it using multiple open-source LLM engines, demonstrating a **2×** performance improvement over standard LLM methods and a **10×** improvement over prior baselines. An ablation study further confirms the contribution of each component, highlighting the impact of code slicing, multi-role reasoning, and filtering mechanisms.

Contributions. Our key contributions include being the first to: **❶ Repository-Level and Merge-Request Granularity:** Elevating automated code review from snippet-level tasks to repository-wide and merge-request (pull-request) granularity. **❷ Integration with Real-World DevOps Workflows:** Deploying automation into a practical online review system with more practical and objective evaluation metrics beyond text similarity. **❸ Validation on Industry-Scale Defects:** Demonstrating effectiveness on real-world, high-impact failures in industry-level codebases instead of synthetic test data. **❹ Code-Review-Oriented LLM Framework:** Designing a specialized framework leveraging code slicing, multi-role collaboration, and filtering mechanisms, achieving substantial improvement in code review performances.

2. Background: Code Review Automation

Automating code review is crucial for maintaining software quality by identifying critical bugs early. The goal is to detect severe issues in new merge requests and provide

necessary comments. In 2022, company reports showed that 30% of severe P1+ incidents (asset losses exceeding \$350,000) and 24.04% of P4+ incidents stemmed from low-level faults due to inadequate reviews. Even in 2024, change-related core failures accounted for 67% of incidents, with code change-related graded incidents comprising 19.54%, highlighting the urgent need for effective automated review tools. These tools help ensure thorough, compliant reviews, reducing defect risks.

To understand reviewer needs, we surveyed a super reviewer group, summarizing findings in Section D. Background on code slicing and multi-role systems, key techniques in our work, is introduced in Sections E and F.

3. Proposed Approach

3.1. Overview

Figure 2 illustrates our decoupling process of code review automation architecture: **1) Code Slicing:** Extracting code from the diff hunk within repository context (Section 3.2); **2) Multi-role Code Review System:** Employing a multi-role system to conduct reviews and compile the results (Section 3.3); **3) Redundancy Comment Filter Mechanism:** Filtering out redundant and irrelevant comments to avoid nitpicks and hallucinations (Section 3.4); **4) Line Number Localization:** Ensuring precise identification of code lines where issues occur (Section 3.5). To evaluate the automation, we construct a dataset from historical fault reports, simulating real-world merge requests that introduced defects (Section 3.6).

3.2. Code Slicing

Previous work used method-level or diff-level code snippets as independent inputs. However, new code is integrated into a larger codebase during reviews, and understanding the structural context is crucial. We developed a code slicing process that integrates multiple slicing strategies, selectable based on the analysis needs. To avoid redundant slices, we use a caching mechanism to enhance efficiency.

The pseudo code of our slicing algorithms is presented in

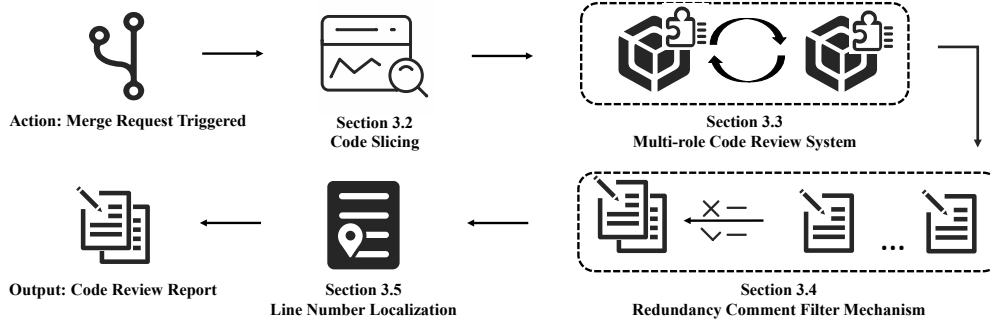


Figure 2. An overview of our automated code review workflow.

Section G. Initially, the repository is cloned, and the merge request commit is checked out. A static analysis tool is then applied to generate abstract syntax trees (ASTs), which serve as the foundation for our slicing process. Based on data dependencies and control flow analysis, one or more of the following four optional slicing algorithms may be applied: **1) Original Diff:** The basic code diff without transformations, capturing essential changes in the commit. **2) Parent Function:** Locates the smallest parent function containing the changes, providing functional context. **3) Left Flow:** Tracks the flow of all left-hand values (L-values) in the function and control structures, focusing on the lifecycle of variables. **4) Full Flow:** Extends Left Flow by tracing right-hand values (R-values) and collecting the signatures of callee functions, offering coverage of variable usage and modifications.

3.3. Multi-role Code Review System

Our multi-role code review system involves four key roles: Reviewer, Meta-Reviewer, Validator, and Translator. These roles collaborate to enhance the accuracy and efficiency of the review process. The system design is illustrated in Figure 3, and we detail the roles and their processes below.

1 Reviewer: Reviews each code snippet generated by the code slicing algorithm (Section 3.2) and provides detailed comments on potential issues in a predefined format. **2 Meta-Reviewer:** Aggregates comments from multiple Reviewers, filtering and sorting them based on predefined thresholds. It merges common issues across reviews. **3 Validator:** Validates and refines the merged comments, re-scores them, and ensures that only comments exceeding a certain threshold are retained. **4 Translator:** Translates the final comments into the required language for multinational teams, ensuring proper formatting for direct integration into the development environment. Each role is integrated with Chain-of-Thought technique, as detailed in Section H.

3.4. Redundancy Comment Filter Mechanism

LLMs often produce an overwhelming number of comments, many of which are either nitpicks or hallucinations. To mitigate this issue, we implemented a Redundancy Comment Filter Mechanism to reduce the number of irrelevant comments.

Our filtering mechanism, integrated within the multi-role system (Section 3.3), operates by answering three key questions for each comment: **Q1: Is this comment a nitpick?** Typical nitpicks include excessive code comments, handling unnecessary edge cases, or overly complex error handling. **Q2: Does the comment identify a fake problem (i.e., a non-existent bug)?** For example, if the comment flags a function call to a known reliable internal library, null pointer checks are considered irrelevant. **Q3: How critical is the issue identified by this comment?** Minor issues, like missing comments, are less severe than potential core dumps or infinite loops.

Each question is rated on a scale from 1 to 7, with 1 indicating a nitpick, fake problem, or minimal issue, and 7 indicating a severe and real issue. The scoring scale (1 to 7) is inspired by other related work (McAleese et al., 2024). We chose this scale to enable a fine-grained and manageable distinction. These scores form the basis of the filtering process throughout the review workflow.

Coarse Filtering and Sorting by Reviewer. During the review process, the Reviewer LLMs score each comment based on Q1-Q3. Comments with Q1 or Q2 scores of 4 or below are discarded. This specific threshold was established heuristically to enhance interpretability and has been validated by developer feedback during internal piloting. The remaining comments are then sorted based on their Q3 score and truncated to the Top-N comments.

Fine Filtering and Sorting by Meta-Reviewer. The Meta-Reviewer further refines the filtered comments by merging those flagged by multiple Reviewers and removing comments mentioned by only one Reviewer.

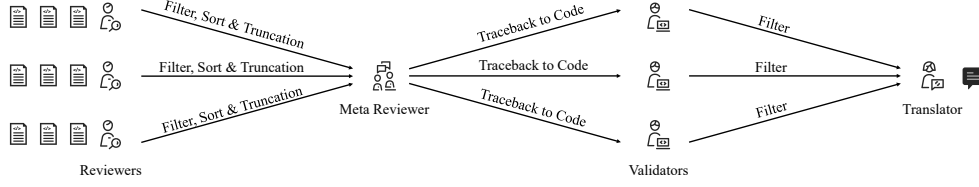


Figure 3. The multi-role system for automating code review.

Validation and Re-scoring by Validators. Validators then re-score the comments by revisiting the original code snippets and applying the same Q1-Q3 criteria. A secondary filter is applied, ensuring that only the most relevant and critical comments proceed to translation and integration into the development platform.

Integration with the Multi-role System. The filtered comments are processed by the remaining multi-role components, including translation (if necessary) and final submission to the development platform. This multi-stage process ensures that the delivered comments are both relevant and concise, minimizing redundancy and false alarms. The heuristic approach to threshold definition described herein was chosen to prioritize generalizability, interpretability, and mitigate overfitting in this study. While providing a robust baseline, exploring adaptive or machine-learned thresholds remains a valuable direction for future enhancement to achieve more nuanced filtering.

3.5. Line Number Localization

A key challenge overlooked in prior work is the precise localization of comments within the code. Unlike code summarization tasks, code reviews require pinpointing specific lines of code where issues are identified. Without this information, developers face inefficiencies in verifying and addressing comments. For example, the change-involved function has 94.54 lines of code in average based on our statistics, missing line localization can result in significant delays for developers.

We propose a code formatting approach inspired by Aider(Gauthier, 2024), tailored for code review tasks. As shown in Table 1, the format includes an operation label (indicating whether a line is kept, added, or deleted), the line number, and the code content. For non-contiguous code lines, ellipses are used to indicate omissions.

Table 1. Code formatting with line position information.

linenumber {kept code line}	Represents lines that remain unchanged.
-linenumber {deleted code line}	Indicates lines that have been removed.
+linenumber {added code line}	Marks newly added lines.
... ...	Indicates the omission of non-essential lines.

3.6. Offline Validation

To systematically assess the performance of our system, we developed a dataset curated from the company’s fault report platform. Each case in this dataset corresponds to an issue that resulted in actual company losses. For each reported fault, we trace back to the merge request that introduced the fault and its subsequent fixing merge request. Using these, we generate ideal reference comments containing details such as affected files, specific lines of code, fault location, root cause, suggested fix, example code, and issue category. The motivation for conducting such validation is illustrated in Section I.

4. Evaluation Design

4.1. Research Questions

We define the following research questions (RQs) to guide our evaluation, whose detailed illustrations are in Section J:

RQ1: How does the overall performance of our framework compare with previous works?

RQ2: How do code slicing algorithms impact the performance of the framework?

RQ3: How do the different components of the multi-role system impact the performance of our framework?

RQ4: How does the redundancy comment filter mechanism address nitpicks and hallucinations?

RQ5: How does the representation of line number position information impact overall performance and line number localization success rate?

4.2. Dataset and Studied Models

The primary goal of code review is to prevent problematic code from being merged into the target branch. To simulate real-world code review scenarios, we collected data from a company’s core framework team, which is responsible for the production code of the short video recommendation core service. This data was gathered using fault reports recorded on an online platform. These cases come from four repositories and involve total 4,090 developers. By analyzing these reports, we traced the merge requests (MRs) that introduced

the issues and examined the specific commits to reproduce the code snapshots. The detailed statistics are presented in Section K.

Our framework supports multiple LLM engines. To mitigate security risks, we only studied open-source models that can be deployed locally. We exclusively selected large instructed models due to the complex human-instruction-based tasks in our workflow. The final list of models includes: LLaMA-3.1 (70B), Qwen2 (72B), Command R+ (104B), Mistral-large-2407 (123B), and LLaMA3.1 (405B). The reasons for not selecting other models are outlined in Section L.

4.3. Metrics

In accordance with the real-world developer expectations discussed in Section D, we evaluate performance at the merge request (MR) level using four metrics, with their formal definitions provided in Section M:

- ❶ **Key Bug Inclusion (KBI)**: Assesses the model’s ability to recall critical issues that could lead to tangible losses.
- ❷ **False Alarm Rate (FAR)**: Captures the proportion of irrelevant or erroneous comments, with two variants (FAR_1 for all MRs and FAR_2 for MRs where key bugs are recalled).
- ❸ **Comprehensive Performance Index (CPI)**: Balances the completeness of key issue detection (KBI) and precision ($100 - FAR$), analogous to the F1-score. It is also computed in two variants (CPI_1 and CPI_2).
- ❹ **Line Localization Success Rate (LSR)**: Measures the accuracy of line-level references by checking whether comments point to the correct code lines.

4.4. Baselines and Experimental Setups

Since our framework focuses on C++, we selected state-of-the-art baselines that support this language: **CodeReviewer** (Li et al., 2022b): A T5 model pre-trained for code review tasks and then fine-tuned. **CCT5** (Lin et al., 2023): A T5 model pre-trained on CodeChangeNet, then fine-tuned. **LLaMA-Reviewer** (Lu et al., 2023): A large LLM fine-tuned for code review tasks based on the LLaMA. **DISCOREV** (Ben Sghaier & Sahraoui, 2024): A T5 model enhanced via cross-task knowledge distillation for code review. The detailed experimental setups of our framework and baselines are presented in Section N.

5. Evaluation Results

5.1. RQ1. Comparison with Baselines

We evaluated the performance of our framework on the fault merge request dataset, comparing it with several baseline approaches. Our framework was tested with different large language model (LLM) engines. Our main experiments primarily utilized a homogeneous setup, employing the same

Table 2. Overall performance comparison of our framework using different LLM engines and baseline models. LLM engines marked with * are quantized. “Val” indicates if Validator role was used.

Model	Val	KBI↑	FAR ₁ ↓	CPI ₁ ↑	FAR ₂ ↓	CPI ₂ ↑
<i>Baselines</i>						
CodeReviewer	—	0.00	97.78	0.00	—	—
CCT5	—	2.22	97.58	2.32	90.91	3.57
LLaMA-Reviewer	—	2.22	97.62	2.30	92.86	3.39
DISCOREV	—	0.00	97.78	0.00	—	—
<i>Ours (Left Flow)</i>						
LLaMA3.1 (70B)	w/o	20.00	84.42	17.52	66.54	25.03
	w	2.22	37.04	4.29	66.67	4.17
Qwen2 (72B)	w/o	40.00	91.21	14.42	83.57	23.29
	w	26.67	90.63	13.87	81.52	21.83
Command R+ (103B)	w/o	20.00	92.99	10.38	76.08	21.78
	w	4.44	85.00	6.86	62.50	7.95
Mistral-2407 (123B)	w/o	26.67	91.07	13.38	74.85	25.89
	w	26.67	87.07	17.41	68.18	29.01
LLaMA3.1 (405B)*	w/o	31.11	87.81	17.51	67.98	31.56
	w	20.00	75.37	22.07	43.52	29.54
<i>Ours (Full Flow)</i>						
LLaMA3.1 (70B)	w/o	13.33	88.22	12.51	61.67	19.78
	w	0.00	46.67	0.00	—	—
Qwen2 (72B)	w/o	42.22	90.99	14.86	83.91	23.30
	w	28.89	91.73	12.86	79.06	24.28
Command R+ (103B)	w/o	15.56	94.22	8.43	77.14	18.51
	w	8.89	76.30	12.93	58.33	14.65
Mistral-2407 (123B)	w/o	28.89	90.68	14.09	75.44	26.55
	w	28.89	86.11	18.76	67.30	30.68
LLaMA3.1 (405B)*	w/o	31.11	89.41	15.80	73.10	28.86
	w	20.00	77.96	20.97	67.59	24.73

LLM across all roles. This approach was chosen to isolate and clearly assess whether a single, powerful model could effectively address key challenges in code review. Recognizing the practical importance and potential benefits of diverse model deployments, we also conducted extended comparison experiments with heterogeneous LLM assignments for reviewer and validator roles. These experiments, detailed in Appendix O, show that strategic combinations, such as pairing a strong validator with a smaller reviewer, can achieve comparable or even superior performance while potentially optimizing resource usage.

For baselines, since they do not prioritize comments, we evaluated their comments based on whether they passed their respective “quality estimation” filters, which assess whether a code snippet requires a comment. The results are in Table 2.

The results indicate that our framework significantly outperforms the baselines by a factor of 10x across most key metrics, such as key bug inclusion (KBI) and comprehensive performance index (CPI). This marked improvement is likely due to our framework’s end-to-end approach to code review automation, which addresses the key challenges of the task and introduces strategies specifically designed to tackle each challenge.

Among the LLM engines tested in our primary setup, LLaMA3.1-405B demonstrated the best overall performance, which aligns with the general scaling laws of language models where capability often increases with parameter count on complex tasks such as code review. However,

our evaluations (detailed in Table 2) also included more compact LLMs. These results show that certain smaller models, particularly those with strong inherent reasoning capabilities, can still achieve competitive performance within our framework. This finding is particularly relevant given the industry trend towards increasing ‘capacity density’ in newer architectures, where smaller models are progressively narrowing the performance gap. While the largest models may provide peak effectiveness, these observations suggest that a range of LLMs can be effectively utilized, allowing for a balance between performance and computational resource demands, a point further explored in our heterogeneous model assignments (Appendix O).

Summary of RQ1. Our framework surpasses baseline approaches significantly (up to 10x on KBI/CPI), thanks to its end-to-end design. LLaMA3.1-405B stands out among tested engines, highlighting the role of model capability. Investigations into heterogeneous LLM combinations also suggest the potential for optimized deployments. (See Appendix T.1 for the extended conclusion.)

5.2. RQ2. Effectiveness of Code Slicing

We tested the four code slicing algorithms described in Section 3.2: Original Diff, Parent Function, Left Flow, and Full Flow. It is important to clarify that while our framework does not employ an explicit Retrieval-Augmented Generation (RAG) pipeline, our code slicing mechanism is designed with a RAG-aligned objective. Specifically, it serves a similar purpose to RAG by strategically retrieving and providing the LLM with only the most relevant contextual code ‘slices’ from the broader codebase. This process aims to focus the model on pertinent information, thereby enhancing its reasoning and effectiveness in the code review task. Our focus in this section is on *KBI* and *CPI*₁, as these metrics indicate how input content affects the maximum recall capability of LLMs for code review.

The experiments were structured to evaluate the comments generated by the large language models under different conditions, including all comments, comments after applying a coarse filter, and top-k ranked comments (based on scores from Q3). We also tested multi-reviewer settings, where the meta-reviewer merges the comments, and validator settings, where validators further refine the comments. The average results are shown in Table 3, based on the LLaMA3.1-405B-AWQ-Int4 LLM engine. To provide further insight into the variability of these results, the minimum and maximum values for each reported metric across the three runs are detailed in Appendix R.

The results reveal that using only the diff or parent function is less effective, while more detailed slicing (Left Flow and Full Flow) improves performance, especially in key bug inclusion. Surprisingly, Left Flow performs better than

Full Flow, likely due to the large language model’s reduced capability when provided with longer contexts, which can cause distraction. This finding supports our assumption that providing targeted and relevant code context is critical for maximizing LLM performance in code review tasks, an observation consistent with the principles underpinning RAG systems where curated information significantly enhances model outputs.

During our analysis of the recalled merge requests (MRs), we found another interesting pattern. Although some slicing algorithms perform worse overall, each algorithm uniquely succeeds in specific cases. This means that each slicing strategy provides valuable context in certain situations. Figure 4 presents a Venn diagram showing the union and differences among the key bugs recalled by each slicing algorithm under the “All” and “+Meta Reviewer” settings. Notably, Left Flow and Full Flow recall most, with significant overlap, but almost each method also uniquely recalls some.

This phenomenon mirrors how human reviewers operate—expanding their focus to different levels of granularity, such as inspecting parent functions or understanding variable usage in different contexts. Some defects are easier to spot in one context, while others require a different view. Therefore, a combination of various slicing strategies might be a promising direction.

Summary of RQ2. Left Flow and Full Flow significantly improve key bug inclusion and overall performance compared to simpler slicing. Left Flow often outperforms Full Flow, possibly because shorter context helps maintain focus. Notably, each slicing approach has exclusive successes, suggesting that combining them could further improve detection. (See Appendix T.2 for the extended conclusion.)

5.3. RQ3. Effectiveness of Multi-role System

To better understand the capabilities of our multi-role system, we conduct experiments on: ❶ Leveraging the non-determinism of large language models; ❷ The self-correction capability (validator); ❸ The chain-of-thought (CoT) prompting strategy.

5.3.1. NUMBER OF REVIEWERS

Previous research has shown that the non-determinism of large language models (LLMs) can impact results. Specifically, with a best-of-N sampling approach, smaller LLMs can sometimes match or surpass larger models. Since our framework includes a multi-reviewer scenario, where a meta-reviewer merges comments from multiple reviewers, we conduct experiments to assess whether increasing the number of reviewers improves performance.

The results in Table 4 show that increasing the number of reviewers from one to three improves *KBI* but also

Table 3. Impact comparison of different code slicing algorithms on key bug inclusion (KBI) and the comprehensive performance index (CPI), based on LLaMA3.1-405B-AWQ-Int4. Experiments for a single reviewer are conducted three times to compute the average. “All” represents all comments generated by the reviewer; “Coarse filter” refers to filtering using Q1 and Q2 scores during generation; “Top-k” denotes truncated comments sorted by Q3 scores; “+Meta Reviewer” and “+Validator” settings are evaluated under Top-5 truncation.

Code Slicing Algorithms	Single Reviewer										Multi Reviewers			
	All		Coarse Filter		Top-10		Top-5		Top-3		+ Meta Reviewer		+ Validator	
	$KBI \uparrow$	$CPI_1 \uparrow$	$KBI \uparrow$	$CPI_1 \uparrow$	$KBI \uparrow$	$CPI_1 \uparrow$	$KBI \uparrow$	$CPI_1 \uparrow$	$KBI \uparrow$	$CPI_1 \uparrow$	$KBI \uparrow$	$CPI_1 \uparrow$	$KBI \uparrow$	$CPI_1 \uparrow$
Original Diff	23.70	5.71	17.04	12.70	16.30	12.75	14.81	12.48	11.11	10.90	13.33	5.24	11.11	10.46
Parent Function	31.85	5.52	22.22	15.18	17.04	13.57	14.81	13.40	8.15	9.63	20.00	11.01	11.11	10.81
Left Flow	37.04	9.77	33.33	12.80	32.59	13.94	25.93	14.26	17.78	12.77	31.11	17.51	20.00	22.07
Full Flow	39.26	9.67	32.59	11.95	31.85	13.18	25.93	13.90	13.33	10.23	31.11	15.80	20.00	20.97

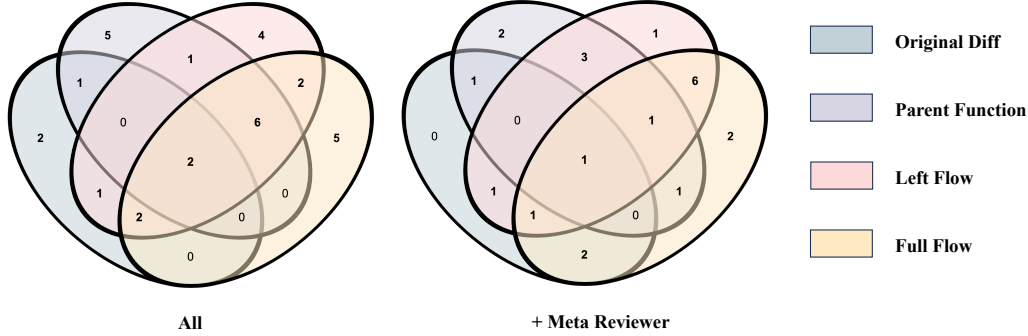


Figure 4. Venn diagram of recalled key bugs identified by different code slicing algorithms. The “All” setting represents all comments, while the “+Meta Reviewer” setting denotes multi-reviewer comments merged by the meta-reviewer. To analyze per-category performance, a breakdown across logic, security, and performance-related bugs is shown in Appendix P.

Table 4. Impact of increasing the number of reviewers from one to three. The “+Meta Reviewer” setting represents the meta-reviewer merging the reviewers’ comments, while the “+Validator” setting denotes the validator refining the comments after the meta-reviewer. All settings use Top-5 truncation of reviewer comments.

Processing Stage	Reviewer Num	$KBI \uparrow$	$FAR_1 \downarrow$	$CPI_1 \uparrow$	$FAR_2 \downarrow$	$CPI_2 \uparrow$
<i>Original Diff</i>						
+ Meta Reviewer	1	8.89	86.15	10.83	69.17	13.80
	3	13.33	96.74	5.24	75.56	17.25
+ Validator	1	4.44	76.59	7.47	73.33	7.62
	3	11.11	90.11	10.46	71.00	16.07
<i>Parent Function</i>						
+ Meta Reviewer	1	15.56	80.37	17.36	73.81	19.52
	3	20.00	92.41	11.01	73.15	22.92
+ Validator	1	6.67	63.70	11.26	55.56	11.59
	3	11.11	89.48	10.81	65.33	16.83
<i>Left Flow</i>						
+ Meta Reviewer	1	26.67	83.26	20.57	70.56	27.99
	3	31.11	87.81	17.51	67.98	31.56
+ Validator	1	11.11	69.26	16.32	23.33	19.41
	3	20.00	75.37	22.07	43.52	29.54
<i>Full Flow</i>						
+ Meta Reviewer	1	22.22	78.04	22.09	71.17	25.10
	3	31.11	89.41	15.80	73.10	28.86
+ Validator	1	15.56	71.78	20.06	61.43	22.17
	3	20.00	77.96	20.97	67.59	24.73

leads to higher FAR_1 and FAR_2 , which negatively affect CPI_1 and CPI_2 in the “+Meta Reviewer” setting. However, after introducing the validator, the performance for three reviewers significantly improves in terms of CPI_1 and CPI_2 . While more reviewers boost KBI , they also increase false alarms, making the validator essential to overall performance.

Summary of RQ3.1. Increasing the number of reviewers lifts key bug inclusion but raises false alarms. A validator mitigates these alarms, implying a trade-off between coverage and precision. (See Appendix T.3 for extended conclusions.)

5.3.2. SELF-CORRECTION ABILITY OF LLMs

In our framework, the validator refines and validates generated comments to correct hallucinations. Table 5 shows that the validator lowers FAR_1 and FAR_2 but also reduces KBI , indicating a trade-off between precision and recall. Our analysis suggests such erroneous rejections of valid comments by validators primarily stem from factors including context propagation from earlier pipeline stages, minor inaccuracies in comment positioning, occasional model input token limits, and inherent scoring variances.

Summary of RQ3.2. Self-correction (validator) reduces false alarms but can inadvertently discard critical bug-

Table 5. The self-correction ability of LLMs through the Validator role. “w/o” denotes without Validator, “w/” denotes with Validator.

Validator Status	KBI↑	FAR ₁ ↓	CPI ₁ ↑	FAR ₂ ↓	CPI ₂ ↑
<i>Original Diff</i>					
w/o	13.33	96.74	5.24	75.56	17.25
w/	11.11	90.11	10.46	71.00	16.07
<i>Parent Function</i>					
w/o	20.00	92.41	11.01	73.15	22.92
w/	11.11	89.48	10.81	65.33	16.83
<i>Left Flow</i>					
w/o	31.11	87.81	17.51	67.98	31.56
w/	20.00	75.37	22.07	43.52	29.54
<i>Full Flow</i>					
w/o	31.11	89.41	15.80	73.10	28.86
w/	20.00	77.96	20.97	67.59	24.73

Table 6. Impact of Chain-of-Thought (CoT) on the framework, presenting paired slicing algorithm comparisons. “SR” denotes Single Reviewer, “MR” denotes Multi Reviewers. All multi-reviewer settings use three reviewers and Top-5 truncation.

Stage	CoT	KBI↑	FAR ₁ ↓	CPI ₁ ↑	KBI↑	FAR ₁ ↓	CPI ₁ ↑
<i>Original Diff</i>							
SR - All	w/o	20.74	97.08	5.12	31.11	96.91	5.60
	w/	23.70	96.74	5.71	31.85	96.98	5.52
SR - Top-5	w/o	17.04	89.17	13.05	16.30	94.57	8.03
	w/	14.81	89.11	12.48	14.81	87.67	13.40
MR - Meta	w/o	15.56	82.74	16.36	17.78	89.09	13.52
	w/	13.33	96.74	5.24	20.00	92.41	11.01
MR - Val	w/o	6.67	63.89	11.26	11.11	75.19	15.35
	w/	11.11	90.11	10.46	11.11	89.48	10.81
<i>Left Flow</i>							
SR - All	w/o	34.81	95.16	8.49	40.00	94.40	9.83
	w/	37.04	94.36	9.77	39.26	94.48	9.67
SR - Top-5	w/o	20.74	92.37	11.10	20.74	91.94	11.50
	w/	25.93	90.15	14.26	25.93	90.43	13.90
MR - Meta	w/o	17.78	88.74	13.79	26.67	82.81	20.90
	w/	31.11	87.81	17.51	31.11	89.41	15.80
MR - Val	w/o	11.11	75.56	15.28	6.67	68.33	11.01
	w/	20.00	75.37	22.07	20.00	77.96	20.97

detecting comments. Balancing these factors is crucial. (See Appendix T.3 for extended conclusions.)

5.3.3. EFFECTIVENESS OF CHAIN-OF-THOUGHT

We compared our specified CoT approach with free-form reasoning. Table 6 shows that CoT prompts often excel in complex slicing tasks (Left Flow, Full Flow), but in simpler tasks (Original Diff, Parent Function), free-form can be just as good or better.

Summary of RQ3.3. CoT prompting is especially beneficial in complex contexts. For simpler code slices, the model may perform well without explicit CoT guidance. As more powerful reasoning models, such as GPT-O1 and DeepSeek-R1, emerge, the advantage of specified CoT over free-form reasoning may further diminish. (Appendix T.3)

5.4. RQ4. Effectiveness of Comment Filter Mechanism

The comment filter mechanism includes ❶ Coarse reviewer filter, ❷ Top-k truncation, ❸ Meta-reviewer filter, and ❹ Validator validation. Table 7 shows that in flow-based slic-

Table 7. The KBI , FAR_1 , and CPI_1 results for different code slicing algorithms utilizing our filtering mechanism. This table illustrates the impact of sequential filter stages, including different Top-k truncation values ($k=10, 5, 3$) for single-reviewer paths. For the multi-reviewer path results shown here (+Meta Reviewer, +Validator), Top-k is set to 5. A comprehensive discussion of Top-k sensitivity, covering both single-reviewer variations and multi-reviewer settings, is presented in Appendix S.

Reviewer	Filter / Trunc	KBI↑	FAR ₁ ↓	CPI ₁ ↑	KBI↑	FAR ₁ ↓	CPI ₁ ↑
<i>Original Diff</i>							
Single	All	23.70	96.74	5.71	31.85	96.98	5.52
	Coarse Filter	17.04	89.72	12.70	22.22	88.17	15.18
	Top-10	16.30	89.43	12.75	17.04	88.53	13.57
	Top-5	14.81	89.11	12.48	14.81	87.67	13.40
	Top-3	11.11	89.14	10.90	8.15	88.15	9.63
Multi	+ Meta	13.33	96.74	5.24	20.00	92.41	11.01
	+ Validator	11.11	90.11	10.46	11.11	89.48	10.81
<i>Left Flow</i>							
Single	All	37.04	94.36	9.77	39.26	94.48	9.67
	Coarse Filter	33.33	92.03	12.80	32.59	92.65	11.95
	Top-10	32.59	91.13	13.94	31.85	91.68	13.18
	Top-5	25.93	90.15	14.26	25.93	90.43	13.90
	Top-3	17.78	90.00	12.77	13.33	91.60	10.23
Multi	+ Meta	31.11	87.81	17.51	31.11	89.41	15.80
	+ Validator	20.00	75.37	22.07	20.00	77.96	20.97

ing (Left Flow, Full Flow), adding these filters sequentially decreases FAR_1 and improves CPI_1 . In simpler slicing (Original Diff, Parent Function), only the coarse filter proves particularly effective, likely due to limited context causing more hallucinations. A comprehensive sensitivity analysis of the Top-k truncation hyperparameter k —detailing its impact on single-reviewer paths with various k values (as presented in Table 7) and an extended analysis within our multi-reviewer framework—is provided in Appendix S.

Summary of RQ4. Our comment filter significantly reduces false alarms and improves performance in more detailed slicing methods. In simpler slicing, the coarse filter stage is the most impactful step. (See Appendix T.4 for the extended conclusion.)

5.5. RQ5. Line Number Position

Line number localization is crucial for real-world applications. We tested three formats: **No**: No line position information is provided; **Relative**: Code is provided with a separate list containing relative line positions; and **Inline**: Position information is integrated directly into the code using the format in Table 1.

Table 8 shows that providing line number information (especially inline) significantly improves performance and localization success rate (LSR).

Summary of RQ5. Embedding line numbers inline yields the highest performance and LSR, likely because it helps the model anchor comments to specific lines accurately. (See Appendix T.5 for the extended conclusion.)

Table 8. Impact of line number position information. “All” represents the average of all comments generated by reviewers, while “+Meta Reviewer” denotes the multi-reviewer workflow with three reviewers and Top-5 truncation. LSR (Line Success Rate) measures whether LLMs provide valid lines, regardless of correctness.

Position	KBI↑	FAR↓	CPI↑	FAR↓	CPI↑	LSR↑
<i>“All” Setting</i>						
No	30.37	95.66	7.58	93.12	11.17	90.54
Relative	42.96	94.60	9.58	92.75	12.32	92.69
Inline	37.04	94.36	9.77	90.66	14.79	91.11
<i>“+ Meta Reviewer” Setting</i>						
No	17.78	90.70	12.21	72.71	21.53	–
Relative	17.78	93.52	9.50	76.04	20.41	–
Inline	31.11	87.81	17.51	67.98	31.56	–

6. Related Work

Code review comments play a crucial role in maintaining software quality, leading to significant research efforts in automating this process. Early studies, such as Gupta & Sundaresan (2018), employed retrieval-based methods, utilizing LSTM models to match new code snippets with historical changes to recommend comments. Siow et al. (2020) advanced this approach by incorporating attention mechanisms to capture semantic nuances more effectively.

With the advent of deep learning, the focus shifted towards automated comment generation. Pioneering efforts by Tufano et al. (2021; 2022) introduced models trained on diverse datasets, including technical texts and code snippets. Subsequent innovations included specialized models such as CodeReviewer (Li et al., 2022b), which leveraged pre-training on code review data, and AUGER (Li et al., 2022a), which used review tags to streamline the task. Another approach, CommentFinder (Hong et al., 2022), presented an efficient retrieval-based model tailored to new code. More recently, LLaMA-Reviewer (Lu et al., 2023) trained large language models specifically for code review tasks, and DISCOREV (Ben Sghaier & Sahraoui, 2024) improved performance by applying cross-task knowledge distillation across successive tasks, and Yu et al. (2024b) focused on fine-tuning LLMs to improve both the accuracy and comprehensibility of automated code reviews. Alongside these advancements in direct comment generation, recent studies have also explored the application of LLMs to other related aspects of the software development lifecycle, such as enhancing code reviewer recommendation (Wang et al., 2024) and automating commit message generation (Tao et al., 2024), underscoring the expanding utility of large models in diverse software engineering contexts.

Despite these advances, previous works have oversimplified the code review process by treating it as a set of snippet-level code-comment pairs. These approaches typically split merge requests into independent snippets and framed the task as a one-to-one neural machine translation (NMT) problem, converting code into natural language. While innova-

tive, this approach provides a limited and idealized view of code review, often evaluated with text similarity metrics, such as BLEU or ROUGE, which do not fully capture the expectations of real-world developers for finding defects.

In practice, code review is more complex, evaluated at the level of entire merge requests of repository codebases rather than individual code-comment pairs. The focus on text similarity fails to consider the broader context, including how comments address the full scope of changes in a MR. Although contributing valuable insights, these studies fall short of replicating the holistic, real-world workflow.

7. Conclusion

Motivated by the limitations of prior research that oversimplified code review automation and fell short of practical applications, we explored the complete automation pipeline within a real-world company. We identified and addressed key challenges such as capturing relevant code context, improving key bug inclusion (KBI), reducing false alarm rates (FAR), and integrating human-centric workflows. Our approach introduces four code slicing algorithms, a multi-role LLM framework, a comment filtering mechanism, and a prompt format with inline line number localization. Evaluations on real-world data demonstrated that we significantly outperforms existing methods, achieving up to a 10x improvement in the comprehensive performance index (CPI) over previous baselines.

Key insights include: ❶ Flow-based slicing (Left Flow and Full Flow) provided better context and outperformed simpler methods. ❷ Increasing the number of reviewers improved KBI but required validation to manage false alarms effectively. ❸ The validator role reduced hallucinations but slightly lowered KBI, highlighting a trade-off between precision and recall. ❹ Chain-of-thought guidance proved more valuable in complex slicing scenarios. ❺ Inline line number localization enhanced both comment accuracy and localization success rates.

Looking ahead, four key areas for future research are: ❶ Enhancing code slicing algorithms to capture more relevant context, potentially combining different slicing levels. ❷ Refining LLM interactions and enhancing engine LLM capability to improve key bug recall. ❸ Further optimizing the filtering mechanism, including the investigation of adaptive or learned thresholds, to reduce nitpicks and hallucinations more effectively. ❹ Streamlining pipeline to make automation more accessible.

Limitation. We discuss limitations in Section V.

Data Availability. We publicly release our codes at <https://zenodo.org/records/14779175>. Details regarding their open-source status can be found in Section U.

Acknowledgments

This work was supported by the National Key Research and Development Program of China (No. 2023YFB3307202) and the Alliance of International Science Organizations Collaborative Research Program (No. ANSO-CR-KP-2022-03).

Impact Statement

This work advances the practical defect-focused applications of automated code review. We redefine the code review task by shifting from a snippet-level code-to-text formulation to an end-to-end, merge-request-level codebase analysis. Our approach fills previously overlooked sub-tasks in the automation pipeline and introduces more practical and objective metrics, better aligning the process with developers' expectations for defect detection in real-world software development. These refinements not only establish a more comprehensive foundation for future research in automated code review but also offer insights applicable to other software engineering tasks. Furthermore, our generic framework, including the proposed context slicing algorithms, provides a versatile methodology that can inspire broader applications in code intelligence.

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Ackerman, A. F., Buchwald, L. S., and Lewski, F. H. Software inspections: an effective verification process. *IEEE software*, 6(3):31–36, 1989.
- AlOmar, E. A., Venkatakrishnan, A., Mkaouer, M. W., Newman, C., and Ouni, A. How to refactor this code? an exploratory study on developer-chatgpt refactoring conversations. In *Proceedings of the 21st International Conference on Mining Software Repositories*, pp. 202–206, 2024.
- Bacchelli, A. and Bird, C. Expectations, outcomes, and challenges of modern code review. In *2013 35th International Conference on Software Engineering (ICSE)*, pp. 712–721. IEEE, 2013.
- Ben Sghaier, O. and Sahraoui, H. Improving the learning of code review successive tasks with cross-task knowledge distillation. *Proceedings of the ACM on Software Engineering*, 1(FSE):1086–1106, 2024.
- Chen, W., Su, Y., Zuo, J., Yang, C., Yuan, C., Chan, C.-M., Yu, H., Lu, Y., Hung, Y.-H., Qian, C., et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *ICLR*, 2024a.
- Chen, Y., Hu, Z., Zhi, C., Han, J., Deng, S., and Yin, J. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024*, pp. 572–576, New York, NY, USA, 2024b. Association for Computing Machinery. ISBN 9798400706585. doi: 10.1145/3663529.3663801. URL <https://doi.org/10.1145/3663529.3663801>.
- Cheng, X., Wang, H., Hua, J., Xu, G., and Sui, Y. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 30(3):1–33, 2021.
- Fagan, M. Design and code inspections to reduce errors in program development. In *Software pioneers*, pp. 575–607. Springer, 2002.
- Gauthier, P. Aider - ai pair programming in your terminal, 2024. URL <https://github.com/paul-gauthier/aider>. Accessed: 2024-09-12.
- Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N., Wiest, O., and Zhang, X. Large language model based multi-agents: A survey of progress and challenges. In *33rd International Joint Conference on Artificial Intelligence (IJCAI 2024)*. IJCAI, 2024.
- Gupta, A. and Sundaresan, N. Intelligent code reviews using deep learning. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD'18) Deep Learning Day*, 2018.
- Hong, Y., Tantithamthavorn, C., Thongtanunam, P., and Aleti, A. Commentfinder: a simpler, faster, more accurate code review comments recommendation. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 507–519, 2022.
- Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., and Ginsburg, B. Ruler: What's the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024.
- Jiang, T., Wang, Z., Liang, J., Li, C., Wang, Y., and Wang, T. Robustkv: Defending large language models against jailbreak attacks via kv eviction. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. OpenReview, April 2025. URL <https://openreview.net/forum?id=L5godAOC2z>. Poster presentation.

- Kononenko, O., Baysal, O., and Godfrey, M. W. Code review quality: How developers see it. In *Proceedings of the 38th international conference on software engineering*, pp. 1028–1038, 2016.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Li, L., Yang, L., Jiang, H., Yan, J., Luo, T., Hua, Z., Liang, G., and Zuo, C. Auger: automatically generating review comments with pre-training models. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1009–1021, 2022a.
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., and Zhong, Y. Vuldeepecker: A deep learning-based system for vulnerability detection. In *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*. The Internet Society, 2018. URL https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_03A-2_Li_paper.pdf.
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., and Chen, Z. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Transactions on Dependable and Secure Computing*, 19(4):2244–2258, 2021.
- Li, Z., Lu, S., Guo, D., Duan, N., Jannu, S., Jenks, G., Majumder, D., Green, J., Svyatkovskiy, A., Fu, S., et al. Automating code review activities by large-scale pre-training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1035–1047, 2022b.
- Liang, J., Wang, Y., Li, C., Zhu, R., Jiang, T., Gong, N., and Wang, T. Graphrag under fire. *arXiv preprint arXiv:2501.14050*, 2025.
- Lin, B., Wang, S., Liu, Z., Liu, Y., Xia, X., and Mao, X. Cct5: A code-change-oriented pre-trained model. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*, 2023.
- Lin, C.-Y. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pp. 74–81, 2004.
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems*, 6:87–100, 2024.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- Lu, J., Yu, L., Li, X., Yang, L., and Zuo, C. Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pp. 647–658. IEEE, 2023.
- Lu, J., Li, Z., Shen, C., Yang, L., and Zuo, C. Exploring the impact of code review factors on the code review comment generation. *Automated Software Engineering*, 31(2):71, 2024.
- Lu, J., Li, X., Hua, Z., Yu, L., Cheng, S., Yang, L., Zhang, F., and Zuo, C. Deepcrceval: Revisiting the evaluation of code review comment generation. In *International Conference on Fundamental Approaches to Software Engineering*, pp. 43–64. Springer, 2025.
- Ma, L., Yang, W., Xu, B., Jiang, S., Fei, B., Liang, J., Zhou, M., and Xiao, Y. Knowlog: Knowledge enhanced pre-trained language model for log understanding. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, pp. 1–13, 2024a.
- Ma, Z., Chen, A. R., Kim, D. J., Chen, T.-H., and Wang, S. Llmpraser: An exploratory study on using large language models for log parsing. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13, 2024b.
- Marjamäki, D. Cppcheck - static analysis of c/c++ code, 2024. URL <https://github.com/danmar/cppcheck>. Accessed: 2024-09-12.
- McAleese, N., Pokorny, R. M., Uribe, J. F. C., Nitishinskaya, E., Trebacz, M., and Leike, J. Llm critics help catch llm bugs, 2024. URL <https://arxiv.org/abs/2407.00215>.
- Organization, P. Flask - the python micro framework for building web applications, 2024. URL <https://github.com/pallets/flask>. Accessed: 2024-09-12.
- Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pp. 311–318, 2002.

- Ray, P. P. Chatgpt: A comprehensive review on background, applications, key challenges, bias, ethics, limitations and future scope. *Internet of Things and Cyber-Physical Systems*, 3:121–154, 2023.
- Rigby, P. C. and Bird, C. Convergent contemporary software peer review practices. In *Proceedings of the 2013 9th joint meeting on foundations of software engineering*, pp. 202–212, 2013.
- Rigby, P. C., German, D. M., and Storey, M.-A. Open source software peer review practices: a case study of the apache server. In *Proceedings of the 30th international conference on Software engineering*, pp. 541–550, 2008.
- Rigby, P. C., German, D. M., Cowen, L., and Storey, M.-A. Peer review on open-source software projects: Parameters, statistical models, and theory. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 23(4): 1–33, 2014.
- Sadowski, C., Söderberg, E., Church, L., Sipko, M., and Bacchelli, A. Modern code review: a case study at google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice*, pp. 181–190, 2018.
- Schäfer, M., Nadi, S., Eghbali, A., and Tip, F. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 2023.
- Shan, Q., Sukhdeo, D., Huang, Q., Rogers, S., Chen, L., Paradis, E., Rigby, P. C., and Nagappan, N. Using nudges to accelerate code reviews at scale. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 472–482, 2022.
- Siow, J. K., Gao, C., Fan, L., Chen, S., and Liu, Y. Core: Automating review recommendation for code changes. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 284–295. IEEE, 2020.
- Sommerville, I. Software engineering 9th edition. *ISBN-10*, 137035152:18, 2011.
- Tao, W., Zhou, Y., Wang, Y., Zhang, H., Wang, H., and Zhang, W. Kadel: Knowledge-aware denoising learning for commit message generation. *ACM Transactions on Software Engineering and Methodology*, 33(5):1–32, 2024.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Tufano, R., Pascarella, L., Tufano, M., Poshyvanyk, D., and Bavota, G. Towards automating code review activities. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 163–174. IEEE, 2021.
- Tufano, R., Masiero, S., Mastropaolo, A., Pascarella, L., Poshyvanyk, D., and Bavota, G. Using pre-trained models to boost code review automation. In *Proceedings of the 44th International Conference on Software Engineering*, pp. 2291–2302, 2022.
- Tufano, R., Martin-Lopez, A., Tayeb, A., Dabić, O., Haiduc, S., and Bavota, G. Deep learning-based code reviews: A paradigm shift or a double-edged sword? *arXiv preprint arXiv:2411.11401*, 2024.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Votta Jr, L. G. Does every inspection need a meeting? In *Proceedings of the 1st ACM SIGSOFT Symposium on Foundations of Software Engineering*, pp. 107–114, 1993.
- Wang, L., Zhou, Y., Zhuang, H., Li, Q., Cui, D., Zhao, Y., and Wang, L. Unity is strength: Collaborative llm-based agents for code reviewer recommendation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pp. 2235–2239, 2024.
- Yang, X., Kula, R. G., Yoshida, N., and Iida, H. Mining the modern code review repositories: A dataset of people, process and product. In *Proceedings of the 13th International Conference on Mining Software Repositories*, pp. 460–463, 2016.
- Yu, L., Lu, J., Liu, X., Yang, L., Zhang, F., and Ma, J. Pscvfinder: a prompt-tuning based framework for smart contract vulnerability detection. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pp. 556–567. IEEE, 2023.
- Yu, L., Chen, S., Yuan, H., Wang, P., Huang, Z., Zhang, J., Shen, C., Zhang, F., Yang, L., and Ma, J. Smart-llama: Two-stage post-training of large language models for smart contract vulnerability detection and explanation. *arXiv preprint arXiv:2411.06221*, 2024a.
- Yu, L., Huang, Z., Yuan, H., Cheng, S., Yang, L., Zhang, F., Shen, C., Ma, J., Zhang, J., Lu, J., and Zuo, C. Smart-llama-dpo: Reinforced large language model for explainable smart contract vulnerability detection. *Proceedings of the ACM on Software Engineering*, 2(ISSTA), July 2025. ISSN 2994-970X. doi: 10.1145/3728878. Licensed under Creative Commons Attribution 4.0 International License.